Università degli Studi di Perugia
Dipartimento di Matematica e Informatica

Tirocinio in Informatica

Analisi famiglie di ransomware con CapeV2

Anno Accademico 2024-2025

Indice

1 Introduzione **3**

2 Configurazione ed utilizzo **6**

2.1 Platform 7
2.2 Platform hypervisor 7
2.3 Sandbox Host/Host machine 8
2.4 Sandbox Hypervisor 8
2.5 Sandbox Guest/Victim machine 8
2.6 Configurazione 8
2.7 Avvio 9
2.8 Sottomissione malware 10
2.9 Analisi 12
2.9.1 Cape API 16

3 Panoramica sui malware **18**

3.1 Contesto attuale 18
3.2 Tipologie principali di malware 20
3.2.1 Virus 20
3.2.2 Worm 20
3.2.3 Trojan 20
3.2.4 KeyLogger 20
3.2.5 Ransomware 21

4 Analisi automatica di ransomware **23**

Capitolo 1

Introduzione

CapeV2 è una malware sandobox opensource che fornisce un ambiente di analisi automatizzata per file malevoli. Una sandbox è uno strumento per eseguire file sospetti o malevoli in un ambiente isolato andando a monitorarne il comporamento. Capev2 non è altro che un'estensione di Cuckoo v1 il quale aveva le seguenti caratteristiche:

- Analisi del comportamento di un eseguibile, basandosi sulle API di sistema chiamate;
- Cattura dei file creati, modificati e/o cancellati durante l'esecuzione;
- Cattura del traffico di rete salvata poi in un file .pcap;
- Classificazione dei malware in base alle signature;
- Screenshot del desktop durante l'esecuzione del malware;
- Dump della memoria del sistema bersaglio.

Capev2 estende queste funzioni con:

- Malware detection tramite le "YARA Rules" le quali fanno uno scan dei file e della memoria per rilevare attività malevole;
- Estrazione dei payload;
- Interfaccia web per l'utilizzo del software.

Capev2 è stato pensato per analizzare principalmente:

- Eseguibili Windows generici;
- File DLL;
- Documenti PDF;
- Documenti Microsoft Office;
- Altro.

Il software è utilizzabile da solo oppure può essere integrato in altri framework grazie al suo design modulare. La sandbox Cape consiste in un software centrale che gestisce l'esecuzione e l'analisi del file malevolo. Ogni analisi è lanciata in una virtual machine isolata, l'infrastruttura è formata da una host machine (gestore del software centrale) e una o più guest machine che appunto ospitano l'esecuzione del file malevolo. L'immagine seguente mostra l'architettura:

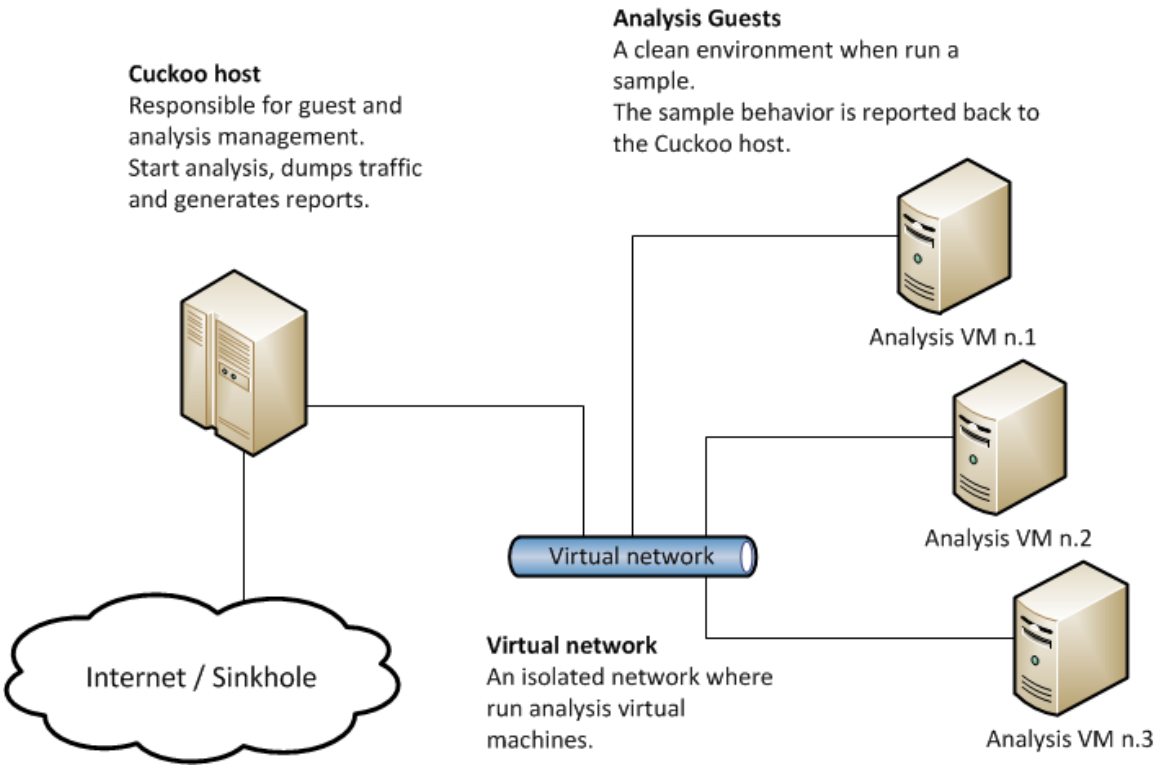

Figura 1.1: Architettura CapeV2

Capitolo 2

Configurazione ed utilizzo

Per configurare CapeV2 ci sono vari metodi ma preferibilmente, per questioni di sicurezza, si predilige quella che è la "Nested Virtualization". Si parte dal sistema operativo iniziale dove viene installato il platform hypervisor (gestore di macchine virtuali) nel quale si crea una macchina virtuale la quale rappresenta l'host machine o sandbox host. Una volta dentro l'host machine si installa il sandbox hypervisor nel quale si installa una macchina virtuale che rappresenta la sandbox guest o victim machine. Infine si installa un'interfaccia di rete isolata condivisa da host machine e victim machine. Una volta configurata si possono iniziare ad analizzare i malware. Anche se CapeV2, secondo la documentazione, può essere usato su un qualsiasi sistema operativo, esso è stato progettato per sistemi GNU/Linux come Ubuntu. Mentre per quanto riguarda la victim machine, CapeV2 è stato pensato per analizzare malware su Windows 10/11.

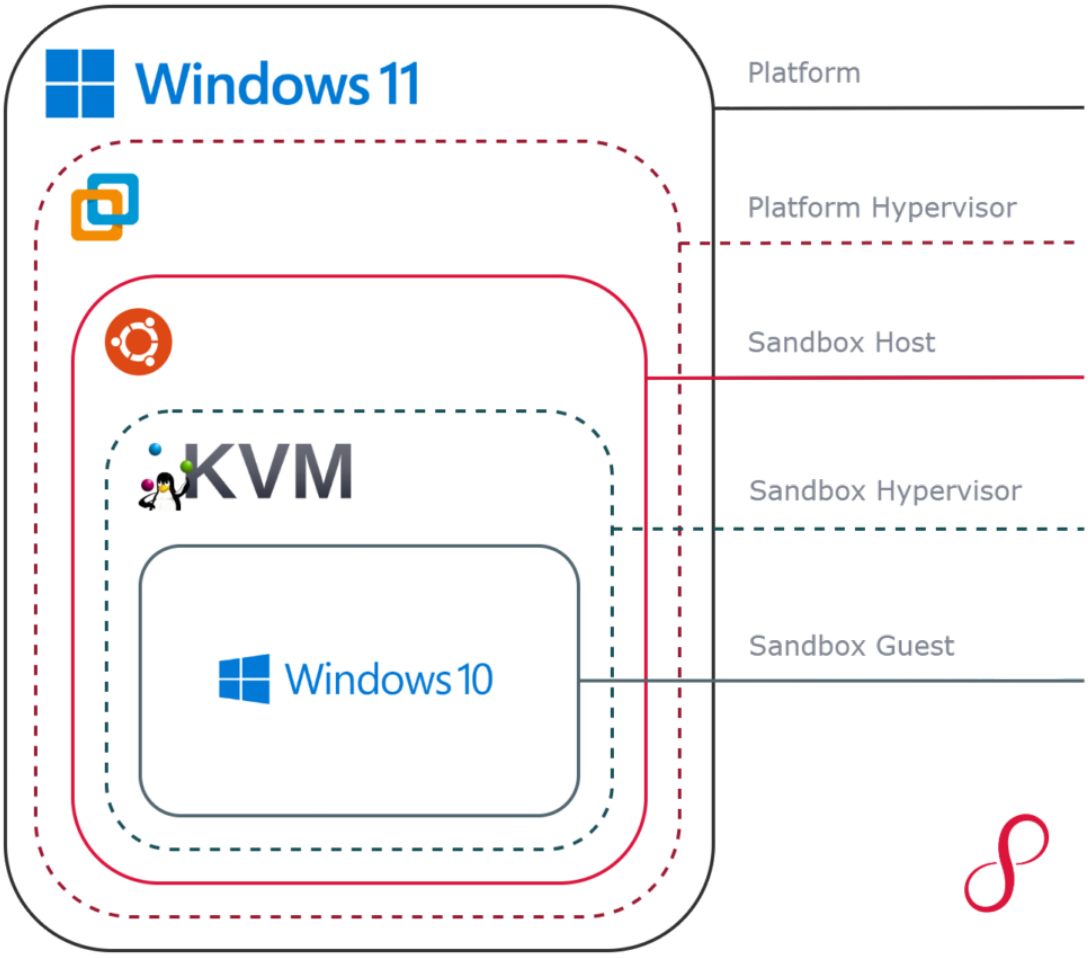

Figura 2.1: Virtualizzazione annidata

2.1 Platform

La platform utilizzata nella configurazione è Windows 11.

2.2 Platform hypervisor

Il platform hypervisor installato in Windows 11 è VMware Workstation Pro.

2.3 Sandbox Host/Host machine

Come host machine è stato scelto il sistema operativo Ubuntu LTS. Dove successivamente è stato scaricato ed eseguito lo script di installazione di CapeV2

2.4 Sandbox Hypervisor

Come sandbox hypervisor è stato utilizzato Virt-Manager che utilizza la libreria "libvirt" per interfacciarsi con KVM (kernel-based virtual machine).

2.5 Sandbox Guest/Victim machine

Come sandbox guest è stata scaricata ed installata la victim machine Windows 10.

2.6 Configurazione

1. Dotare la victim machine di tutti i software che un utente "normale" usa, questo perché il malware potrebbe capire di trovarsi in un ambiente di analisi andando ad interrompere parzialmente o totalmente la sua esecuzione;

2. Disattivare Windows Defender, Real-time Protection e il Firewall;

3. Scaricare il Cape Agent che è un programma scritto in Python che permette l'invio dei malware da l'host machine alla victim machine. Al file dell'agent va modificata l'estensione da .py a .pyw permettendo quindi un'esecuzione silenziosa, successivamente va creata una "scheduled task" che esegue l'agent al momento dell'avvio della victim machine;

4. Scaricare Python.

5. L'ultimo passo è quello di creare una rete per mettere in comunicazione l'host machine con la victim machine. Quindi è stata installata nell'host machine un'interfaccia di rete isolata, ovvero una configurazione host-only che permette la comunicazione solo tra victim machine e host machine, con indirizzo IP:

192.168.100.0/24. Alla scheda di rete della victim machine è stato dato l'indirizzo IP: 192.168.100.50/24, successivamente è stata testata la comunicazione tramite il comando "ping". Infine è stato fatto uno snapshot in modo tale che tra l'analisi di un malware e un altro la victim machine risulti sempre "pulita".

```
3: virbr1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
    link/ether 52:54:00:96:a1:8d brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.1/24 brd 192.168.100.255 scope global virbr1
       valid_lft forever preferred_lft forever
```

Figura 2.2: Interfaccia di rete isolata lato host machine

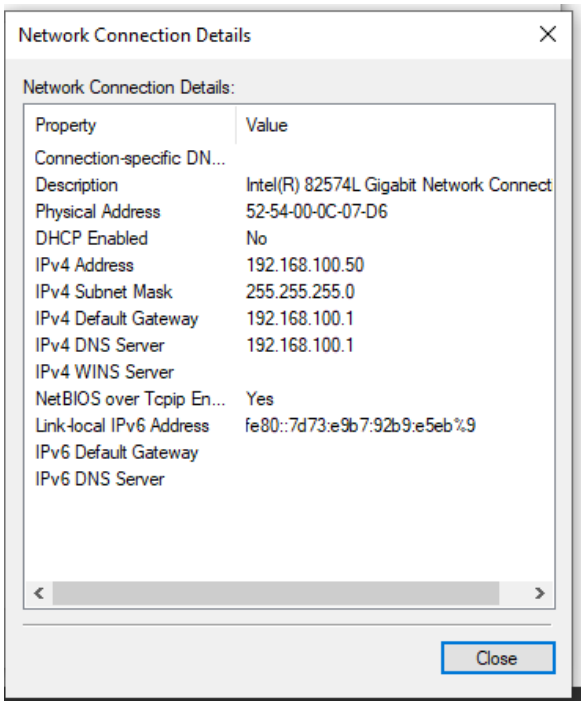

Figura 2.3: Interfaccia di rete isolata lato victim machine

2.7 Avvio

Per avviare Cape si esegue cuckoo.py che si trova in /opt/CAPEv2, il comando è il seguente:

```
sudo -u cape etc/poetry/bin/poetry run python opt/CAPEv2/cuckoo.py
```

Una volta avviato digitare sul browser l'URL:

```
http://localhost:8000
```

In questo modo si aprirà l'interfaccia web di Capev2:

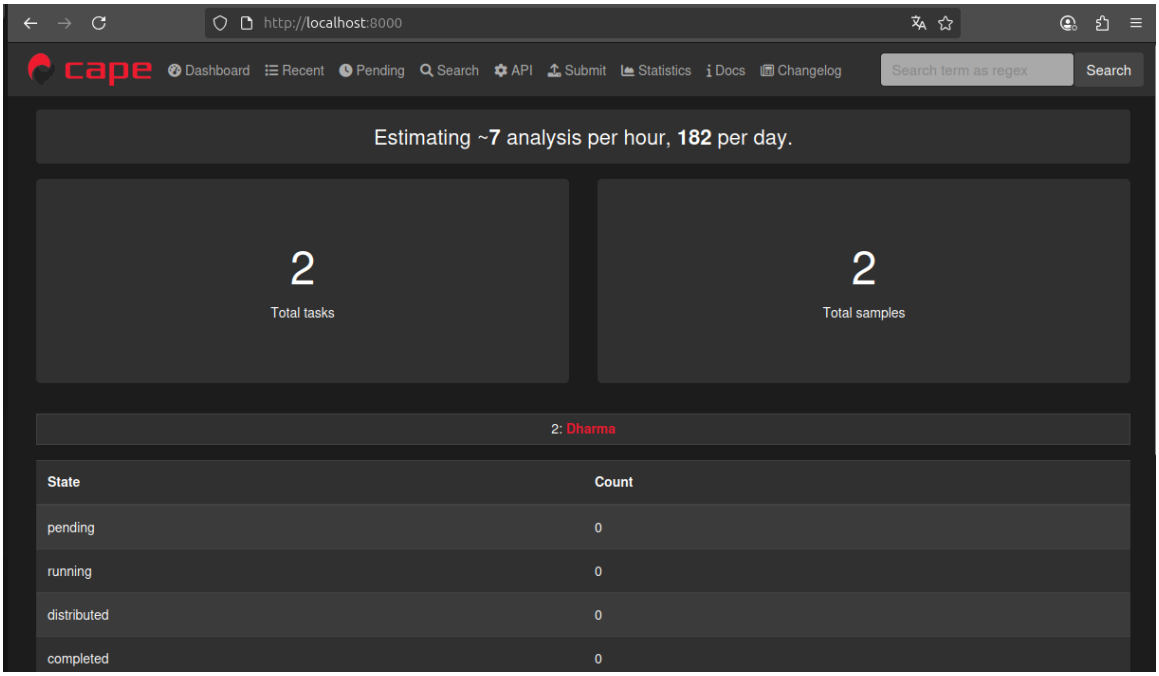

Figura 2.4: Interfaccia web CapeV2

2.8 Sottomissione malware

Per sottoporre un malware all'analisi di Cape bisogna innanzitutto scaricare il sample di un malware, i siti principali dove poterlo fare sono:

- `https://bazaar.abuse.ch/`;
- `https://virusShare.com` ;
- `https://virusTotal.com` ;

Se ad esempio volessimo testare l'esecuzione di un MassLogger, virus della famiglia KeyLogger che effettua un furto dei dati tramite i log degli strumenti di input, possiamo fare nel modo che segue:

1. Scarichiamo il sample di un MassLogger:

Figura 2.5: Sample di un MassLogger

2. Andiamo alla pagina submit dell'interfaccia di Cape e selezioniamo il sample del MassLogger per l'analisi:

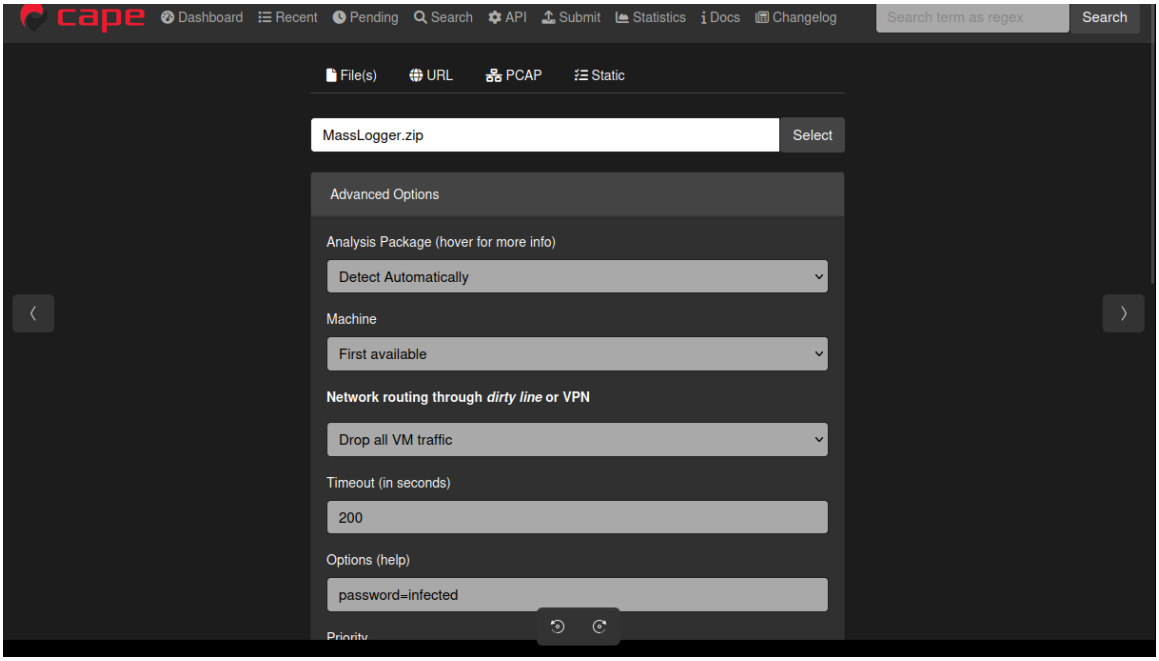

Figura 2.6: Sottomissione del file

Una volta sottomesso il file Cape esegue lo snapshot della victim machine e fa partire l'analisi.

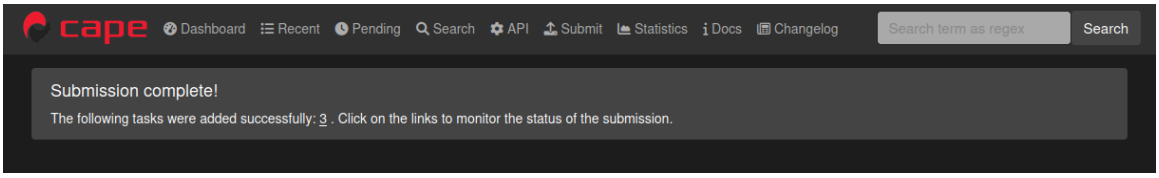

Figura 2.7: Sottomissione avvenuta

2.9 Analisi

Quando l'esecuzione del malware è terminata si hanno a disposizione svariati strumenti per l'analisi del malware tra cui:

- Veloce panoramica: che include i dettagli dell'analisi e del file analizzato, gli screenshot acquisiti nella victim machine durante l'esecuzione del malware, le signatures (patterns univoci che software come Cape utilizzano per il rilevamento dei malware) con la possibilità di visualizzarne i dettagli, i file acceduti, i file modificati, i file letti, i file eliminati, le chiavi di registro lette, le chiavi di registro modificate, le chiavi di registro lette, le API di sistema risolte, i comandi eseguiti, i mutex utilizzati ed infine i servizi avviati.

A process created a hidden window

Terminates another process

A scripting utility was executed

Uses Windows utilities to create a scheduled task

A process attempted to delay the analysis task by a long amount of time.

Touches a file containing cookies, possibly for information gathering

Attempts to schedule tasks using an XML files that doesn't have .xml extensions

Yara detections observed in process dumps, payloads or dropped files

Deletes executed files from disk

Attempts to modify Windows Defender using PowerShell

Harvests credentials from local FTP client softwares

Harvests information related to installed mail clients

regkey: HKEY_CURRENT_USER\Software\Microsoft\Office\16.0\Outlook\Profiles\Outlook\9375CFF0413111d3B88A00104B2A6676
regkey: HKEY_CURRENT_USER\Software\Microsoft\Office\15.0\Outlook\Profiles\Outlook\9375CFF0413111d3B88A00104B2A6676

Executed a process and injected code into it, probably while unpacking

Figura 2.9: Rilevazioni comportamenti anomali tramite le regole YARA

Analysis

Category	Package	Started	Completed	Duration	Options	Log(s)
FILE		2025-07-26 12:38:51	2025-07-26 12:45:56	425 seconds	Show Options	Show Analysis Log

Machine

Name	Label	Manager	Started On	Shutdown On	Route
win10	win10	KVM	2025-07-26 12:38:51	2025-07-26 12:43:53	none

File Details

File Name	**MassLogger.zip**
File Type	**Zip archive data, at least v5.1 to extract, compression method=AES Encrypted**
File Size	**600148 bytes**
MD5	cd7a83ea9cad65bca50ef5c0aa0840f4
SHA1	27307e3566a8a18ef8e3e34a9740a7fe3de8a715

Figura 2.8: Panoramica Veloce

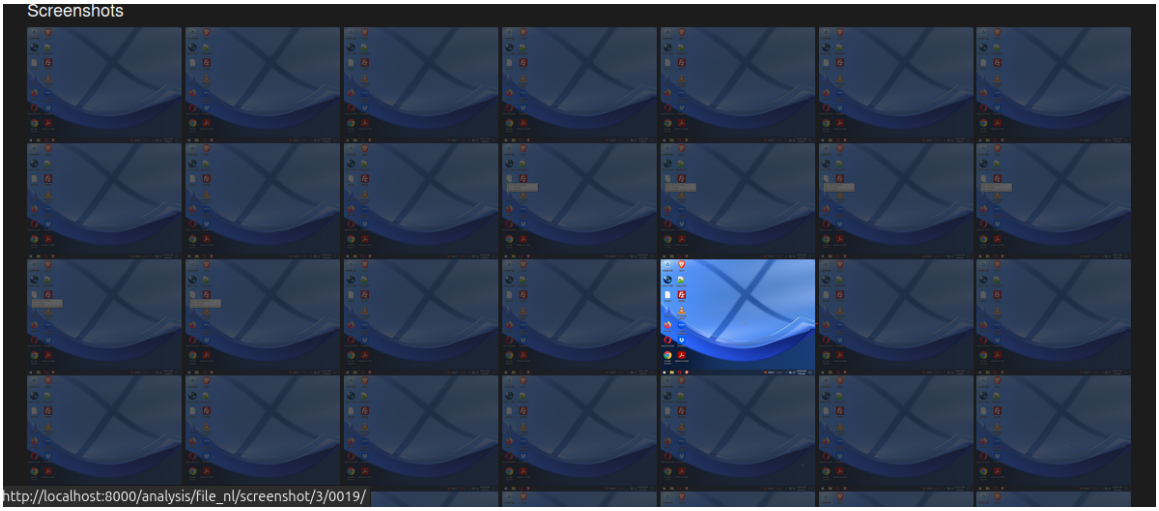

Figura 2.10: Catture dello schermo durante l'esecuzione del malware

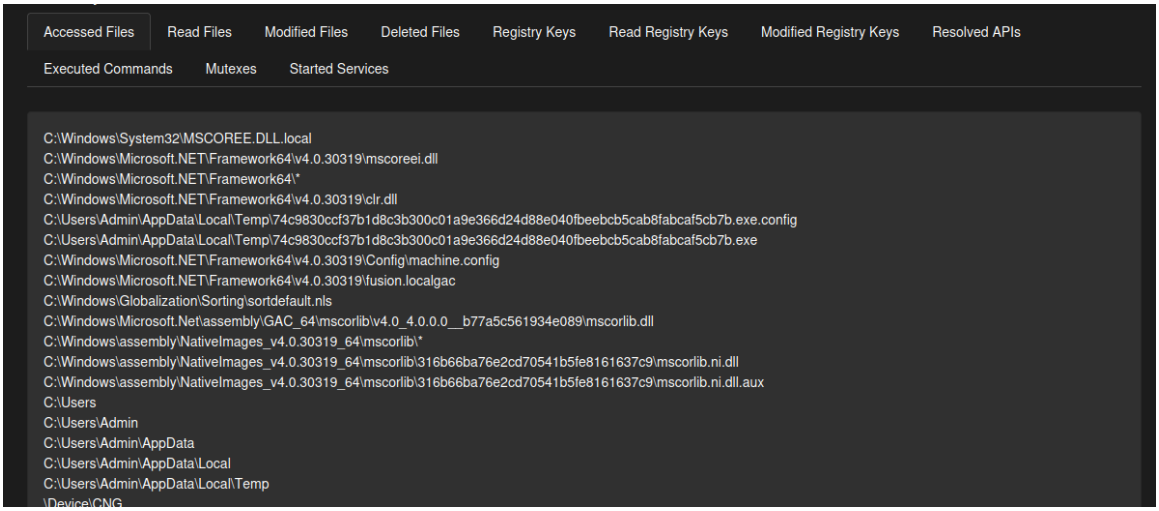

Figura 2.11: File acceduti

- Analisi comportamentale: che include l'albero dei processi da cui è possibile andare a verificare l'origine dei processi con relativi PID e PPID e le API di sistema chiamate con i relativi dettagli come: il thread che le ha chiamate,

gli argomenti e il loro valore di ritorno. Alle API c'è anche l'opportunità di applicare dei filtri.

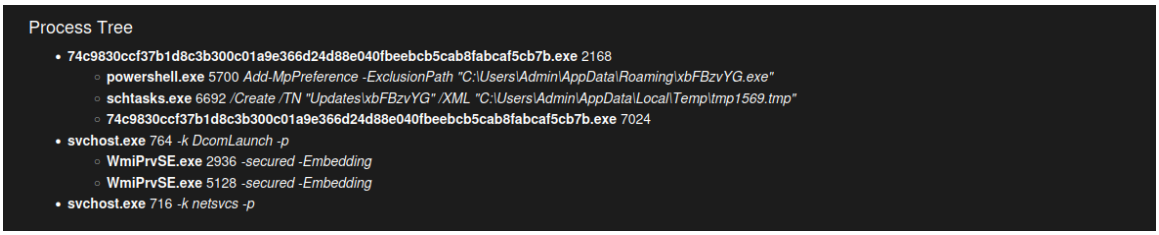

Figura 2.12: Albero dei processi

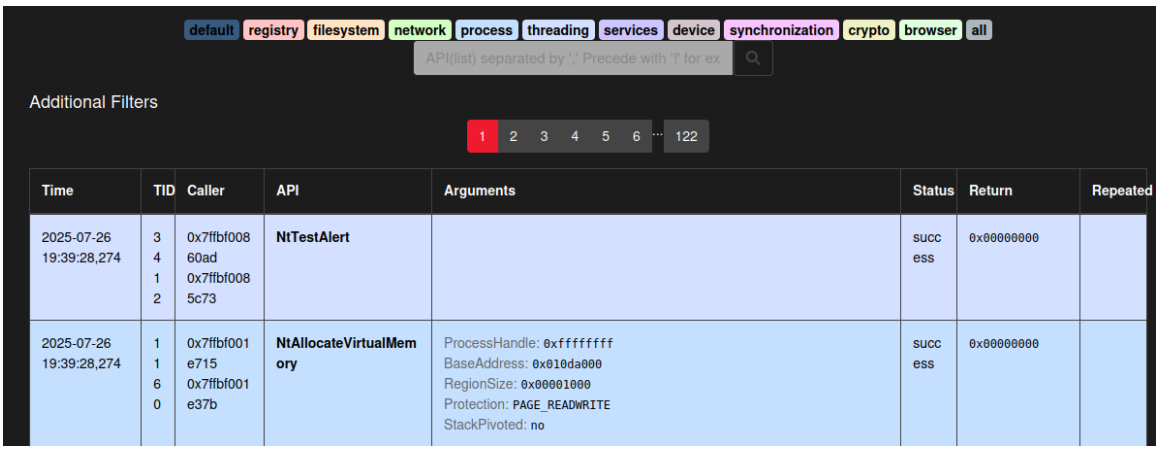

Figura 2.13: API di sistema chiamate

- File "dropped": tutti quei file che il malware durante la sua esecuzione salva sul disco, di solito questi file vengono utilizzati dal malware per compiere altre azioni malevole. Cape offre la possibilità di scaricare questi file, sottometterli ad una nuova analisi oppure permette di visualizzarli.

- Payload: ovvero l'estrazione della componente fondamentale di un malware. Non è altro che la parte di codice entro cui il malware compie le azioni dannose. Cape per i payload offre le stesse funzionalità che offre per i file "dropped";

- File .pcap: durante l'analisi è attivo uno sniffer sulla victim machine che produce un file .pcap il quale può essere sottomesso a strumenti come Wireshark o tcpdump.

2.9.1 Cape API

Cape utilizza delle API per compiere azioni come: sottomissione di file, eliminare un task, scaricare il payload di un malware e molto altro. La lista di queste API è disponibile a http://locahost:8000/apiv2/, andando nel file /opt/CAPEv2/conf/api.conf abbiamo la possibilità di disabilitare o abilitare ognuna di queste API. Visto che si tratta di API RESTful dobbiamo prima installare il framework djangorest tramite:

```
sudo -u cape etc/poetry/bin/poetry run pip install djangorestframework
```

i modi principali per chiamarle sono principalmente due:

1. Da terminale ad esempio (sottomissione di un file all'analisi) :

   ```
   curl -F file=@/path/to/file http://localhost:8000/apiv2/tasks/create/file
   ```

2. Da programma Python (sottomissione di un file all'analisi):

```python
import requests
import json

url = "http://localhost:8000/apiv2/tasks/create/file/"
SAMPLE_FILE ="/home/leonal/Scrivania/malware/Dharma.zip"

parameters = {
    "options": "password=infected"
}

with open(SAMPLE_FILE,"rb") as sample:
    multipart_file = {"file": ("temp_file_name", sample)}
    response = requests.post(url, files=multipart_file,data = parameters)

if response.status_code == 200:
    try:
        data = response.json()
        id_task = data['data']['task_ids'][0]
        print(f"{response.status_code}")
        print(id_task)
    except json.JSONDecodeError:
        print("not valid JSON")
else:
    print("request error")
    print(f"{response.status_code}")
    print(f"{response.text}")
```

Figura 2.14: Sottomissione file con Python

Capitolo 3

Panoramica sui malware

3.1 Contesto attuale

Attualmente all'incirca 560,000 malware vengono rilevati ogni giorno e ne esistono oltre 1 miliardo. In particolar modo 5.33 vulnerabilità emergono ogni minuto, quindi al giorno d'oggi la minaccia continua ad incombere su ogni tipo di organizzazione al mondo. Detto ciò è chiaro come la cybersecurity rimane una preoccupazione fondamentale per le organizzazioni e aziende di ogni tipo. Naturalmente la maggior parte dei malware è progettata principalmente per Windows questo a causa di un insieme di motivi:

- Windows è il sistema operativo più utilizzato al mondo per questo gli sviluppatori preferiscono, anche per motivi economici, creare malware ad hoc per Windows piuttosto che per altri sistemi operativi;

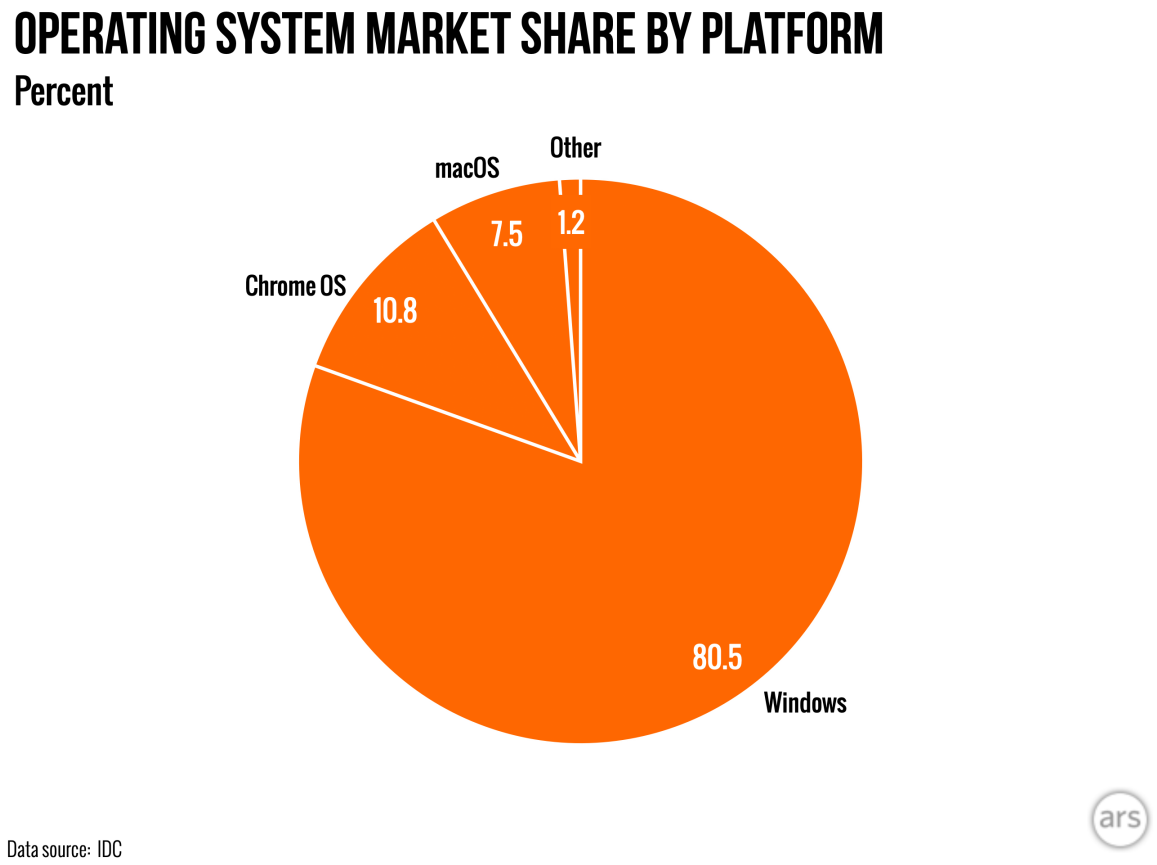

Figura 3.1: Utilizzo dei vari OS

- Storicamente Windows ha avuto più vulnerabilità permettendo ai malware di infettare il sistema e diffondersi più facilmente rispetto ad altri sistemi operativi;
- Windows a differenza di sistemi operativi come MacOS offre agli utenti una maggiore libertà per il download di software, questa è però un'arma a doppio taglio perché se da un lato gli utenti possono disporre di un'infinità di software quest'ultimi possono rappresentare vulnerabilità che possono essere sfruttate dai malware.

3.2 Tipologie principali di malware

3.2.1 Virus

I virus sono la tipologia di malware più comune, si tratta di codice malevolo autoreplicante, si propaga modificando altri programmi nella macchina target e iniettando codice malevolo all'interno di essi. Necessitano l'interazione dell'utente come ad esempio l'esecuzione di un programma o l'apertura di un file infetto. Tipicamente l'infezione avviene tramite il download di file.

3.2.2 Worm

I worm sono programmi che non necessitano l'interazione dell'utente e che si diffondono rapidamente. Possono eseguire payload per eliminare file o creare delle botnet, una botnet non è altro che una rete di computer infetti controllata dallo stesso hacker, a differenza dei virus i worm tipicamente degradano la rete anche solo consumando larghezza di banda. Si propagano all'interno del sistema trovando vulnerabilità. Nel 2022 i worm hanno rappresentato la tipologia di malware più presente. Tipicamente l'infezione avviene tramite e-mail di phishing.

3.2.3 Trojan

Ispirato al "Cavallo di Troia" della mitologia greca, il trojan è un malware che si nasconde dietro un software apparentemente innocuo. Una volta preso il controllo del sistema il trojan può: effettuare furti di dati, accedere alla rete, creare delle botnet e spiare il dispositivo. Anche qui l'infezione solitamente avviene tramite e-mail di phishing.

3.2.4 KeyLogger

Strumenti software e hardware che silenziosamente catturano tutto quello che un utente digita. Rubano dati personali e finanziari per poi trasmetterli ad un hacker, permettendogli di accedere a informazioni riservate.

3.2.5 Ransomware

Un tipologia di malware in rapida crescita sono i ransomware i quali infettano un sistema ne criptano i file e chiedono successivamente al malcapitato un riscatto per la decriptazione dei file. I dati ci dicono che la perdita economica media in seguito all'infezione da parte di un ransomware ammonta a circa 4.65 milioni di USD. Inoltre il modello RaaS (Ransomware as a Service) permette ad un hacker di creare un ransomware andandolo poi a vendere ad altri hacker, questo ha permesso ad alcuni gruppi di ransomware di diffondersi notevolmente. I gruppi di ransomware maggiormente coinvolti in attacchi informatici negli ultimi anni sono:

- Akira: famiglia di ransomware molto attiva che attacca aziende di medie dimensioni, osservato per la prima volta nel 2023;
- LockBit : famiglia di ransomware molto attiva nel 2022, responsabile di oltre 1,700 attacchi negli US, solo nel 2020 si stima abbia guadagnato oltre 91 milioni di USD tramite riscatti. Nel 2023 ha criptato Royal Mail chiedendo 80 milioni di USD per il riscatto e più tardi nello stesso anno ha criptato Taiwan Semiconductor Manufacturing Company (TSMC) chiedendo 70 milioni di USD per il riscatto;
- Dharma (variante Crysis): gli attaccanti che usano questo ransomware fanno uno scan della rete per trovare computer che utilizzano il servizio RPD (remote desktop protocol) con TCP sulla porta 3389. Una volta trovati tentano degli attacchi brute force per acquisirne la password e quando sono dentro il sistema installano il ransomware che procede con la cifratura dei file;
- MedusaLocker : uno dei ransomware più aggressivi operativo sin dal 2021. Utilizza la tattica della doppia estorsione cifrando i file del malcapitato e minacciandone la pubblicazione, solo nel 2025 ha compiuto 430 attacchi in tutto il mondo;
- Rhysida: famiglia di ransomware che ha guadagnato popolarità pubblicando dati sensibili, le sue vittime includono ospedali ed entità governative locali;
- ALPHV/ BlackCat;

- Royal/BlackSuit;
- RansomHub;
- NoEscape;
- BlackBasta;
- WannaCry.

Capitolo 4

Analisi automatica di ransomware

Utilizziamo uno script Python che automatizza l'analisi di una famiglia di ransomware tra le seguenti:

1. Akira (original version) scaricato da `https://bazaar.abuse.ch/sample/06c2a137c31aae5d02`
2. LockBit (versione 4.0) scaricato da `https://bazaar.abuse.ch/sample/25fba0e92d00184dde6`
3. Rhysida (original version) scaricato da `https://bazaar.abuse.ch/sample/1a9c27e5be8c58da1c02fc4245a07831d5d431cdd1a91cd35d2dd0ad62da71cd/`;
4. MarLock (estensione MedusaLocker) scaricato da `https://bazaar.abuse.ch/sample/1bc0575b3fc6486cb2510dac1ac6ae4889b94a955d3eade53d3ba3a92d133281/`;
5. Dharma (estensione Crysis) scaricato da `https://bazaar.abuse.ch/sample/e9bbcfb5d9f42ef0dd75eb435e78d5226087679593893e0c08977694e720cd7a/`.

Per fare ciò verranno utilizzate le API di Cape. Alla fine di ciascuna analisi verrà creata una directory che al suo interno avrà:

- cape_report.txt: contiene i rilevamenti dell'analisi con la relativa YARA Rule utilizzata, i "dropped file" catturati durante l'analisi, le signature con una descrizione breve e il loro livello di gravità ed infine i TTPs (Tactics, Techniques and Procedures);
- files_report.txt: contiene i file letti, modificati ed eliminati all'interno della victim machine durante l'esecuzione del ransomware;

- key_registry_report.txt: contiene le chiavi di registro lette, modificate ed eliminate all'interno della victim machine durante l'esecuzione del ransomware;
- processtree.pdf : immagine che raffigura l'albero dei processi disegnato tramite la libreria Python "Digraph";
- net_capture.pcap: contiene la cattura del traffico di rete tra l'host machine e la victim machine durante l'esecuzione del ransomware.

L'analisi automatizzata inizia chiedendo all'utente quale dei 5 ransomware vuole analizzare:

```
Which ransomware do you want to analyze with Cape?
1. Akira
2. LockBit
3. Rhysida
4. MedusaLocker
5. Dharma
Your choiche:
```

Figura 4.1: Scelta Ransomware

Una volta che l'utente fa una scelta il sample del ransomware viene sottomesso all'analisi utilizzando l'API Cape: http://localhost:8000/apiv2/tasks/create/file/ Poi viene inizializzata una stringa "SAMPLE_FILE" la quale contiene il percorso assoluto del sample, questa stringa verrà ultimata soltanto dopo la scelta dell'utente.

```
url_create_file = 'http://localhost:8000/apiv2/tasks/create/file/'
headers = {'Authorization' : 'Token d256567a66cc7cb3f903389f873277b5a1fe4bb2'}
SAMPLE_FILE = '/home/leonal/Scaricati/'
```

Figura 4.2: Inizializzazione delle stringhe: URL, headers e SAMPLE_FILE

```
Akira = '06c2a137c31aae5d02b4d7df61ffd31f1af9a9e59978f15b3f7265cc751bff1f.zip'
LockBit = '25fba0e92d00184dde662c7d30aff006851dba296daa6f6f82ce797b66789ed2.zip'
Rhysida = '1a9c27e5be8c58da1c02fc4245a07831d5d431cdd1a91cd35d2dd0ad62da71cd.zip'
MedusaLocker = '1bc0575b3fc6486cb2510dac1ac6ae4889b94a955d3eade53d3ba3a92d133281.zip'
Dharma = 'e9bbcfb5d9f42ef0dd75eb435e78d5226087679593893e0c08977694e720cd7a.zip'
```

Figura 4.3: Inizializzazione delle stringhe che poi verranno utilizzate per completare la stringa SAMPLE_FILE

```
while True:
    print('Which ransomware do you want to analyze with Cape?')
    print('1. Akira')
    print('2. LockBit')
    print('3. Rhysida')
    print('4. MedusaLocker')
    print('5. Dharma')


    choiche = input('Your choiche:')
    if not choiche.isdigit():
        print('You must insert a numeric value between 1 and 5.')
        continue

    choiche = int(choiche)
    if choiche < 1 or choiche > 5:
        print('You must insert a value between 1 and 5')
        continue
    break
```

Figura 4.4: Scelta dell'utente

```
if choiche == 1:
    SAMPLE_FILE += Akira
elif choiche == 2:
    SAMPLE_FILE += LockBit
elif choiche == 3:
    SAMPLE_FILE += Rhysida
elif choiche == 4:
    SAMPLE_FILE += MedusaLocker
else:
    SAMPLE_FILE += Dharma
```

Figura 4.5: Concatenazione della stringa rappresentante il ransomware scelto dall'utente con SAMPLE_FILE

Tramite la libreria "requests", viene effettuata una chiamata POST all'endpoint dell'API per avviare l'analisi. Successivamente, utilizzando "subprocess", viene creata la directory dedicata all'analisi del ransomware selezionato dall'utente, all'interno della quale verranno salvati tutti i report generati. Infine, viene avviata la cattura del traffico sull'interfaccia di rete isolata tramite tcpdump.

```
try:
#we need 'with' to avoid the use of close() after the read of file opened through open()
    with open(SAMPLE_FILE, 'rb') as sample:
        params = {
            'package' : '',
            'machine' : '',
            'timeout' : 200,
            'options' : 'password=infected',
            'priority' : 2,
            'platform' : '',
            'tags' : '',
            'custom' : '',
            'memory' : '',
            'enforce_timeout' : '',
            'clock' : ''
            }
        multipart_file = {'file' : (SAMPLE_FILE, sample)}
        response = requests.post(url_create_file, headers = headers, files = multipart_file, data = params)
        data = response.json()
        id_task = data['data']['task_ids'][0]
        subprocess.run(['mkdir', f'cape_analysis_{id_task}'], capture_output = True)
        capture = subprocess.Popen(['tcpdump','-i','virbr1','-w',f'/home/leonal/Scrivania/tirocinio/cape_analysis_
except requests.RequestException as e:
    print(f'request error: {e}')
    exit(1)
except json.JSONDecodeError as e:
    print(f'Not a valid JSON: {e}')
    exit(1)
except FileNotFoundError as e:
    print(f'{SAMPLE_FILE} do not exist: {e}')
    exit(1)
```

Figura 4.6: Avvio dell'analisi, creazione cartella relativa all'analisi avviata e avvio della cattura di rete

Tramite la risposta in formato JSON che il server restituisce prendiamo l'id del task e lo usiamo per fare richieste continue tramite questa API:
http://localhost:8000/apiv2/tasks/status
la quale ci restituisce lo stato di lavorazione dell'analisi in modo da fermare lo script fin quando l'analisi non è completata:

```python
while True:
    try:
        status = requests.get(url_task_status, headers = headers).json()['data']
        print('The status of your task with id:', id_task, 'is ' + status)
        if(status != 'reported'):
            print('Please wait...')
            time.sleep(10)
        else:
            break
    except requests.RequestException as e:
        print(f'Request error : {e}')
        exit(1)
    except json.JSONDecodeError as e:
        print(f'Not valid JSON: {e}')
        exit(1)
```

Figura 4.7: Porzione di codice ripetuta fin quando l'analisi non è terminata

```
Your choiche:5
59
The status of your task with id: 59 is pending
Please wait...
```

Figura 4.8: Terminale durante l'esecuzione dell'analisi

```
The status of your task with id: 59 is running
Please wait...
The status of your task with id: 59 is completed
Please wait...
The status of your task with id: 59 is completed
Please wait...
The status of your task with id: 59 is completed
Please wait...
The status of your task with id: 59 is completed
Please wait...
The status of your task with id: 59 is completed
Please wait...
The status of your task with id: 59 is reported
The analysis is finished.
Reports will be downloaded at /home/leonal/Scrivania/tirocinio/cape_analysis_59
```

Figura 4.9: Analisi completata

Una volta che l'analisi è completata utilizziamo sempre l'id del task per fare altre due richieste al server tramite queste due API:
http://localhost:8000/apiv2/tasks/get/report
http://localhost:8000/apiv2/tasks/get/iocs
Successivamente salviamo le risposte ricevute dal server all'interno delle variabili report e iocs.

```
url_get_report = f'http://localhost:8000/apiv2/tasks/get/report/{id_task}'
url_get_iocs = f'http://localhost:8000/apiv2/tasks/get/iocs/{id_task}/detailed'

try:
    response = requests.get(url_get_report, headers = headers)
    report = response.json()
except requests.RequestException as e:
    print(f'Request error : {e}')
    exit(1)
except json.JSONDecodeError as e:
    print(f'Not valid JSON: {e}')
    exit(1)

try:
    response = requests.get(url_get_iocs, headers = headers)
    iocs = response.json()
except requests.RequestException as e:
    print(f'Request error : {e}')
    exit(1)
except json.JSONDecodeError as e:
    print(f'Not valid JSON: {e}')
    exit(1)
```

Figura 4.10: Salvataggio delle risposte JSON ottenute tramite API all'interno delle varibili "report" e "iocs"

Per disegnare l'albero dei processi utilizziamo le funzione addNode() e addNodeChildren, albero che poi verrà salvato all'interno della directory precedentemente creata.

```
#function to add a principal node
def addNode(process = None):
    if process == None:
        return
    else:
        if not process.get('pid') in nodes_pid:
            label = f'{process.get('name')}\nPID: {process.get('pid')}\nNumber of threads: {len(process.get('thre
            dot.node(f'{process.get('pid')}', label )
            nodes_pid.add(process.get('pid'))
            for child in process['children']:
                addNodeChildren(child,process)
            return
        else:
            return

#function to add a children node
def addNodeChildren(child = None, parent_process = None):
    if not child.get('pid') in nodes_pid:
        child_label = f'{child.get('name')}\nPID: {child.get('pid')}\nPPID: {parent_process.get('pid')}\nNumber o
        dot.node(f'{child.get('pid')}', child_label)
        nodes_pid.add(child.get('pid'))
        dot.edge(f'{parent_process.get('pid')}', f'{child.get('pid')}')
        for c_child in child.get('children'):
            addNodeChildren(c_child, child)
    else:
        return
```

Figura 4.11: Funzioni utilizzate per disegnare l'albero dei processi

```
for process in report['behavior']['processtree']:
    addNode(process)

dot.render(f'cape_analysis_{id_task}/processtree', view = False)
```

Figura 4.12: Uso di dot.render() per salvare l'albero dei processi all'interno della cartella relativa all'analisi

Infine scriviamo i tre file .txt usando le risposte salvate dentro report e iocs.

```
try:
    with open(f'cape_analysis_{id_task}/cape_report.txt' , 'w') as r:

        r.write('DETECTIONS:\n')
        for detection in report['detections']:
            for detail in detection['details']:
                r.write(f'Cape detected {detection.get('family')} using YARA rule: {detail.get('Yara')}\n')

        r.write('\nDROPPED FILES:\n')
        for i,dropped_file in enumerate(report['dropped'], start = 0):
            r.write(f'NO. {i}:\n')
            r.write(f'Name: {dropped_file.get('name')}\nPath: {dropped_file.get('path')}\n\n')

        r.write('SIGNATURES:\n')
        for i,signature in enumerate(report['signatures'], start = 0):
            r.write(f'NO. {i}:\n')
            r.write(f'Name: {signature.get('name')}.\nDescription: {signature.get('description')}\nSeverity level:

        r.write('TTPS:\n')
        for i,ttp in enumerate(report['ttps'], start = 0):
            r.write(f'NO. {i}:\n')
            r.write(f'Signature: {ttp.get('signature')}.\nttps: {ttp.get('ttps')}\n\n')
except Exception as e:
    print(f'Error during report creation: {e}')
```

Figura 4.13: Utilizzo di write() per scrivere un file .txt relativo alle rilevazioni, i file catturati, le signatures e le TTPS

```
try:
    with open(f'cape_analysis_{id_task}/files_report.txt','w') as r:

        r.write('MODIFIED:\n\n')
        for mod in iocs['data']['files']['modified']:
            r.write(f'{mod}\n')

        r.write('\n\nDELETED:\n\n')
        for deleted in iocs['data']['files']['deleted']:
            r.write(f'{deleted}\n')

        r.write('\n\nREAD:\n\n')
        for read in iocs['data']['files']['read']:
            r.write(f'{read}\n')
except Exception as e:
    print(f'Error during File report creation: {e}')
```

Figura 4.14: Utilizzo di write() per scrivere un file .txt relativo ai file modificati, eliminati e letti

```python
try:
    with open(f'cape_analysis_{id_task}/key_registry_report.txt', 'w') as r:

        r.write('MODIFIED:\n\n')
        for mod in iocs['data']['registry']['modified']:
            r.write(f'{mod}\n')

        r.write('\n\nDELETED:\n\n')
        for deleted in iocs['data']['registry']['deleted']:
            r.write(f'{deleted}\n')

        r.write('\n\nREAD:\n\n')
        for read in iocs['data']['registry']['read']:
            r.write(f'{read}\n')
except Exception as e:
    print(f'Error during Key Registy report creation: {e}')
```

Figura 4.15: Utilizzo della funzione write() per scrivere un file .txt relativo alle chiavi di registro modificate, eliminate e lette

Alla fine dell'esecuzione dello script Python avremo una cartella relativa all'analisi appena fatta con all'interno i file citati precedentemente.

Figura 4.16: Cartella relativa all'analisi

Figura 4.17: File creati alle figure: 4.12, 4.13, 4.14, 4.15.

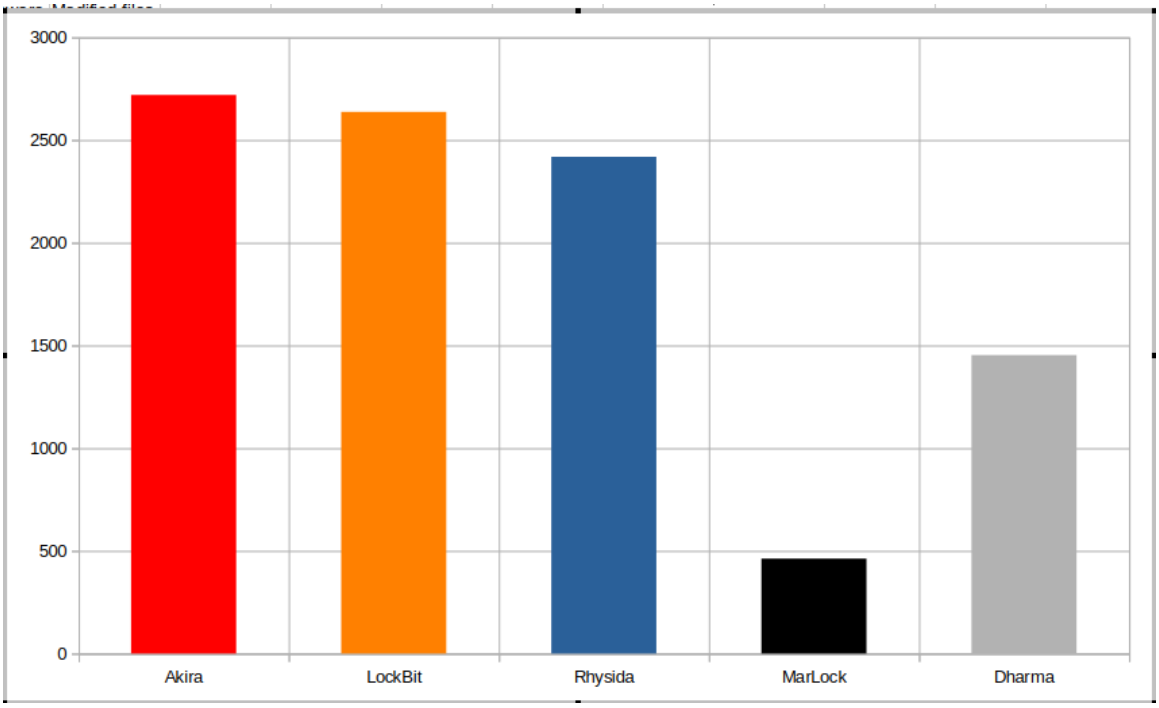

Figura 4.18: File modificati da ogni ransomware

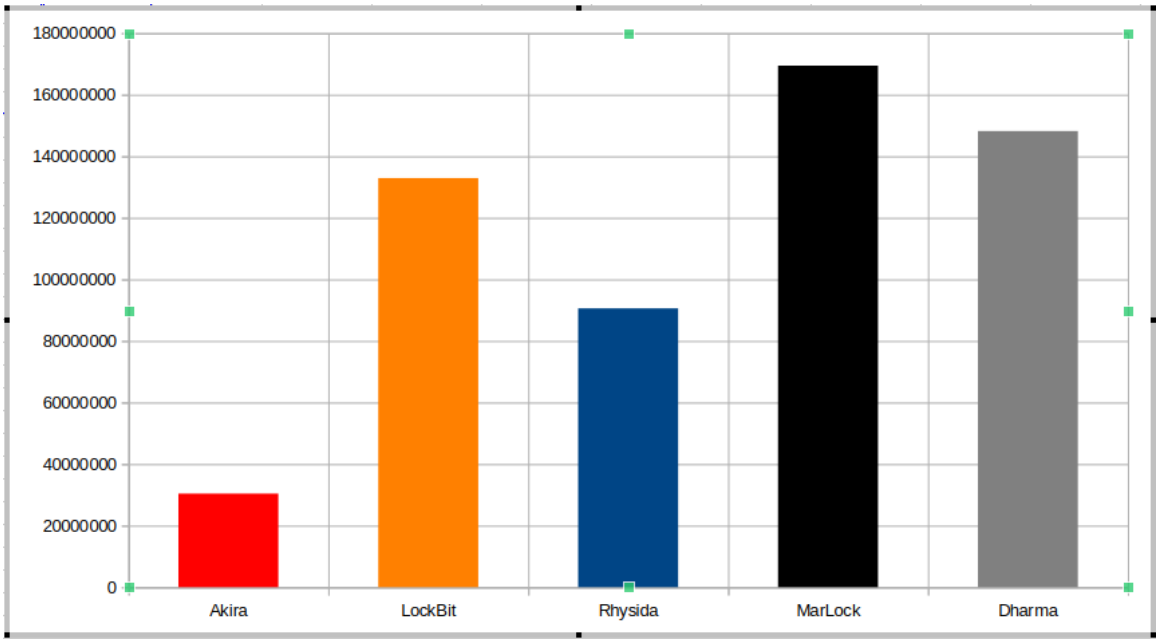

Figura 4.19: Byte in pacchetti scambiati tra host e victim durante l'analisi